

# Data Structures

Complete PYQ Answers | MSE Question Bank | Modules 1, 2 & 3

## MODULE 1

### Q1. Define Data Structure. Explain Types of Data Structures.

A data structure is a systematic way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

#### Types of Data Structures:

| Type       | Description                                                  | Example                          |
|------------|--------------------------------------------------------------|----------------------------------|
| Primitive  | Basic built-in data types directly supported by the language | int, char, float, bool           |
| Linear     | Elements arranged in sequential order                        | Array, Stack, Queue, Linked List |
| Non-Linear | Elements arranged in hierarchical or network order           | Tree, Graph                      |
| Static     | Fixed size determined at compile time                        | Array                            |
| Dynamic    | Size changes dynamically at runtime                          | Linked List                      |

#### Classification Tree:

```
Data Structures
├── Primitive (int, float, char, bool)
└── Non-Primitive
    ├── Linear
    │   ├── Static -> Arrays
    │   └── Dynamic -> Linked List, Stack, Queue
    └── Non-Linear
        ├── Trees (Binary Tree, BST, AVL...)
        └── Graphs
```

### Q2. Explain the Concepts of ADT with Examples.

Abstract Data Type (ADT) is a mathematical model that defines a data type by its behavior (operations) rather than its implementation. It specifies WHAT operations are performed, not HOW they are implemented.

#### Components of ADT:

- Data: the values stored
- Operations: the functions that can be performed on the data

### Example 1 — Stack ADT:

```
Stack ADT:
Data: Collection of elements (LIFO order)
Operations:
  push(x)  -> Insert element at top
  pop()    -> Remove top element
  peek()   -> View top element without removing
  isEmpty() -> Check if stack is empty
  isFull() -> Check if stack is full
```

### Example 2 — List ADT:

```
List ADT:
Data: Ordered sequence of elements
Operations: insert(pos,x), delete(pos), search(x), traverse()
```

*Key Insight: The user does not need to know if the stack uses an array or linked list internally — that is the power of abstraction!*

## Q3. Define Algorithm and List its Properties.

An algorithm is a finite, well-defined sequence of steps or instructions to solve a specific problem or perform a computation.

| Property      | Explanation                                           |
|---------------|-------------------------------------------------------|
| Input         | Zero or more inputs must be supplied to the algorithm |
| Output        | At least one output must be produced                  |
| Definiteness  | Each step must be clear and unambiguous               |
| Finiteness    | Must terminate after a finite number of steps         |
| Effectiveness | Each step must be basic and feasible to execute       |

### Example — Find the largest of two numbers:

```
Step 1: Start
Step 2: Read A, B
Step 3: If A > B, print A else print B
Step 4: Stop
```

## Q4. Analyze Efficiency Using Big O, Big Omega, and Big Theta Notations.

Asymptotic notations describe the time/space complexity of an algorithm as input size  $n$  grows.

## Big O — $O(f(n))$ — Upper Bound (Worst Case)

"The algorithm will never be slower than this."

$f(n) = O(g(n))$  if there exist constants  $c > 0$  and  $n_0$  such that:

$$f(n) \leq c * g(n) \quad \text{for all } n \geq n_0$$

Example: Linear search  $\rightarrow O(n)$  — at worst, checks every element.

## Big Omega — $\Omega(f(n))$ — Lower Bound (Best Case)

"The algorithm will never be faster than this."

$f(n) = \Omega(g(n))$  if:

$$f(n) \geq c * g(n) \quad \text{for all } n \geq n_0$$

Example: Linear search  $\rightarrow \Omega(1)$  — best case finds element at index 0.

## Big Theta — $\Theta(f(n))$ — Tight Bound (Average Case)

"The algorithm is exactly this fast — upper and lower bounds are the same."

$f(n) = \Theta(g(n))$  if:

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

Example: Merge Sort  $\rightarrow \Theta(n \log n)$  always.

| Notation       | Bound Type  | Case         | Meaning            |
|----------------|-------------|--------------|--------------------|
| $O(g(n))$      | Upper Bound | Worst Case   | At most this slow  |
| $\Omega(g(n))$ | Lower Bound | Best Case    | At least this fast |
| $\Theta(g(n))$ | Tight Bound | Average Case | Exactly this rate  |

## Common Complexities (Best to Worst):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

## Q5. Explain Time and Space Complexity.

### Time Complexity

The amount of time an algorithm takes to run as a function of input size  $n$ .

| Case         | Meaning                                      |
|--------------|----------------------------------------------|
| Best Case    | Minimum time with the most favorable input   |
| Average Case | Expected time for random or typical input    |
| Worst Case   | Maximum time with the most unfavorable input |

### Example — Linear Search:

- Best Case:  $O(1)$  — element found at first position
- Average Case:  $O(n/2)$  which simplifies to  $O(n)$
- Worst Case:  $O(n)$  — element not found

## Space Complexity

The amount of memory an algorithm uses, including:

- Fixed Part: code, constants, variables (independent of input size)
- Variable Part: depends on input size (recursion stack, dynamic arrays)

```
Space Complexity = Fixed Space + Variable Space
```

### $O(1)$ Space Example — only one extra variable used:

```
int sum = 0;
for(int i = 0; i < n; i++) sum += a[i];
```

### $O(n)$ Space Example — extra array of size n:

```
int b[n]; // uses n extra memory cells
```

## Q6. Differentiate Between:

### 6.1 Array vs Linked List

| Feature            | Array                          | Linked List                        |
|--------------------|--------------------------------|------------------------------------|
| Memory Layout      | Contiguous memory              | Non-contiguous memory              |
| Size               | Fixed (static)                 | Dynamic (grows/shrinks at runtime) |
| Access             | Random $O(1)$                  | Sequential $O(n)$                  |
| Insertion/Deletion | $O(n)$ — shifting required     | $O(1)$ — at a known node           |
| Extra Memory       | No overhead                    | Extra pointer per node             |
| Cache Performance  | Better (locality of reference) | Poorer                             |

### 6.2 Stack vs Queue

| Feature       | Stack                    | Queue                          |
|---------------|--------------------------|--------------------------------|
| Order         | LIFO — Last In First Out | FIFO — First In First Out      |
| Insertion End | Top only                 | Rear only                      |
| Deletion End  | Top only                 | Front only                     |
| Operations    | push, pop, peek          | enqueue, dequeue               |
| Use Cases     | Undo, recursion, DFS     | Printer queue, BFS, scheduling |

### 6.3 Top-Down vs Bottom-Up Approach

| Feature      | Top-Down                            | Bottom-Up                     |
|--------------|-------------------------------------|-------------------------------|
| Strategy     | Divide big problem into subproblems | Solve small subproblems first |
| Technique    | Recursion + Memoization             | Iteration + Tabulation        |
| Memory Usage | More (stack frames)                 | Less                          |
| Example      | Merge Sort, Recursive Fibonacci     | Dynamic Programming           |

### 6.4 Big O vs Big Omega vs Big Theta

| Notation       | Type        | Condition                                      | Represents   |
|----------------|-------------|------------------------------------------------|--------------|
| $O(g(n))$      | Upper Bound | $f(n) \leq c \cdot g(n)$                       | Worst case   |
| $\Omega(g(n))$ | Lower Bound | $f(n) \geq c \cdot g(n)$                       | Best case    |
| $\Theta(g(n))$ | Tight Bound | $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ | Average case |

# MODULE 2

## Q1. Define List Abstract Data Type (ADT).

A List ADT is an ordered collection of elements where each element has a specific position (index). It supports dynamic insertion, deletion, and traversal operations without exposing internal implementation details.

```
List: [e1, e2, e3, ..., en]
      ^           ^
      index 0      index n-1
```

## Q2. Basic Operations Performed on List ADT.

| Operation      | Description                           | Time Complexity      |
|----------------|---------------------------------------|----------------------|
| insert(pos, x) | Insert element x at position pos      | O(n)                 |
| delete(pos)    | Remove element at position pos        | O(n)                 |
| search(x)      | Find and return position of element x | O(n)                 |
| traverse()     | Visit all elements from start to end  | O(n)                 |
| isEmpty()      | Check if list has no elements         | O(1)                 |
| size()         | Return number of elements in the list | O(1)                 |
| get(pos)       | Return element at given position      | O(1) array / O(n) LL |

## Q3. Array-Based Implementation of List (with Diagram).

In array-based implementation, elements are stored in contiguous memory locations. A size variable tracks the current number of elements.

### Memory Layout:

```
Index:  [ 0 ][ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
Data:   [ 10 ][ 20 ][ 30 ][ 40 ][   ][   ]
              ^
              size=4, max=6
```

### Inserting 25 at position 1 — Shift Right then Insert:

```
Before: [10][20][30][40]
Shift:  [10][20][20][30][40]    <- elements shift right
After:  [10][25][20][30][40]    <- 25 inserted at index 1
```

### C Implementation:

```
void insert(int arr[], int *size, int pos, int val) {
    for(int i = *size; i > pos; i--)
```

```
        arr[i] = arr[i-1];    // shift elements right
    arr[pos] = val;
    (*size)++;
}
```

```
void delete(int arr[], int *size, int pos) {
    for(int i = pos; i < *size-1; i++)
        arr[i] = arr[i+1];    // shift elements left
    (*size)--;
}
```

**Advantages:**

- Simple structure and easy to implement
- Fast random access  $O(1)$  using index

**Disadvantages:**

- Fixed size — cannot grow beyond declared size
- $O(n)$  insertion/deletion due to element shifting

## MODULE 3

### Q1. Define Stack and List its Basic Operations.

Stack is a linear data structure that follows LIFO (Last In, First Out) principle. The element inserted last is the first one to be removed.

#### Stack Diagram:

```
|   |  
| 30 | <- TOP (most recent)  
| 20 |  
| 10 | <- Bottom  
|____|
```

| Operation | Description                                        | Complexity |
|-----------|----------------------------------------------------|------------|
| push(x)   | Insert element x at the top of the stack           | O(1)       |
| pop()     | Remove and return the element from the top         | O(1)       |
| peek()    | View top element without removing it               | O(1)       |
| isEmpty() | Returns true if the stack has no elements          | O(1)       |
| isFull()  | Returns true if the stack has reached its capacity | O(1)       |

### Q2. Define Queue. Explain Different Types of Queue.

Queue is a linear data structure that follows FIFO (First In, First Out) principle. Elements are inserted at the rear and removed from the front.

```
REAR -> [40][30][20][10] -> FRONT  
         enqueue         dequeue
```

#### 1. Simple / Linear Queue

Insert at rear, delete at front. Drawback: memory wastage occurs because front positions become inaccessible after deletion even if space is available at the beginning.

#### 2. Circular Queue

The rear wraps around to connect back to the front, forming a circle. Solves the memory wastage problem of linear queue.

```
[10] -> [20] -> [30]  
  ^       |  
[50] <- [40] <----+
```

#### 3. Double-Ended Queue (Deque)

Insertion and deletion are allowed from both ends (front and rear).

- Input-restricted Deque: Insert only at rear, delete from both ends



- Output-restricted Deque: Delete only from front, insert from both ends

#### 4. Priority Queue

Each element is assigned a priority value. Elements with higher priority are served before those with lower priority, regardless of insertion order. Used in Dijkstra's algorithm and CPU scheduling.

### Q3. Applications of Stack.

| Application               | How Stack is Used                                          |
|---------------------------|------------------------------------------------------------|
| Function Call Stack       | Activation records are pushed on call and popped on return |
| Undo / Redo               | Previous states stored; undo pops the last saved state     |
| Expression Evaluation     | Operands pushed; operators pop operands and push result    |
| Infix to Postfix / Prefix | Operators held in stack until precedence dictates output   |
| Backtracking (DFS / Maze) | Stores previous states to backtrack when path fails        |
| Browser History           | Back button pops the previously visited page URL           |
| Balancing Parentheses     | Opening brackets pushed; closing brackets matched via pop  |
| Reversing a String        | Push all characters, then pop — output is reversed         |

### Q4. ADT to Explain Stack Operations.

```
Stack ADT:
  Structure: Array or Linked List with TOP pointer

  push(x):
    if isFull() -> Error: Stack Overflow
    else -> top++, stack[top] = x

  pop():
    if isEmpty() -> Error: Stack Underflow
    else -> return stack[top], top--

  peek():
    if isEmpty() -> Error
    else -> return stack[top] (no change to top)

  isEmpty():
    return (top == -1)

  isFull():
    return (top == MAX-1)
```

### Q5. Array Representation of Stack (with Diagram).

**Initial state (empty): top = -1**

```
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+
  [0]  [1]  [2]  [3]  [4]
```

**After push(10): top = 0**

```
+---+---+---+---+
| 10 |   |   |   |
+---+---+---+---+
    ^TOP
```

**After push(20) and push(30): top = 2**

```
+---+---+---+---+
| 10 | 20 | 30 |   |
+---+---+---+---+
                ^TOP
```

**After pop(): top = 1**

```
+---+---+---+---+
| 10 | 20 |   |   |
+---+---+---+---+
                ^TOP
```

**C Implementation:**

```
#define MAX 100
int stack[MAX], top = -1;

void push(int x) {
    if(top == MAX-1) printf("Stack Overflow");
    else stack[++top] = x;
}

int pop() {
    if(top == -1) { printf("Stack Underflow"); return -1; }
    return stack[top--];
}

int peek()    { return stack[top]; }
int isEmpty() { return top == -1; }
int isFull()  { return top == MAX-1; }
```

## Q6. Linked List Representation of Stack.

Each node contains data and a pointer to the next node. The TOP pointer always points to the most recently inserted node. No size limit other than available heap memory.

**Stack with elements 10, 20, 30 (30 on top):**

```
TOP
|
[30|*]->[20|*]->[10|NULL]
```

### After push(40):

```
TOP
|
[40|*]->[30|*]->[20|*]->[10|NULL]
```

### C Implementation:

```
struct Node {
    int data;
    struct Node* next;
};
struct Node* top = NULL;

void push(int x) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = top;    // new node points to old top
    top = newNode;         // update top
}

int pop() {
    if(top == NULL) { printf("Underflow"); return -1; }
    int val = top->data;
    struct Node* temp = top;
    top = top->next;
    free(temp);
    return val;
}
```

Advantage: No overflow condition (limited only by heap memory). Size is truly dynamic.

## Q7. Reverse a String/Number Using Stack.

### Algorithm:

- Step 1: Push all characters of the string onto the stack one by one
- Step 2: Pop all characters — they come out in reverse order due to LIFO

### Example — Reverse "HELLO":

```
Input:  H E L L O

Push:   H -> E -> L -> L -> O
Stack (top->bottom): [O] [L] [L] [E] [H]

Pop:    O -> L -> L -> E -> H
Output: O L L E H => "OLLEH"
```

C Code:

```
void reverse(char str[]) {
    int n = strlen(str);
    char stack[100]; int top = -1;

    for(int i = 0; i < n; i++)    // push all characters
        stack[++top] = str[i];

    for(int i = 0; i < n; i++)    // pop to get reverse
        str[i] = stack[top--];
}
```

Q8. Infix to Postfix and Prefix Conversions.

Operator Precedence (High to Low): ^ > \* / % > + -

Postfix Algorithm: Scan left to right. Output operands immediately. For operators, pop stack while top has >= precedence, then push current. At end, pop all.  
Prefix Tip: Reverse the infix, swap ( and ), apply postfix algorithm, then reverse the result.

Expression 1: (A-B)\*(C+D)

| Symbol | Stack | Output  |
|--------|-------|---------|
| (      | (     |         |
| A      | (     | A       |
| -      | (-    | A       |
| B      | (-    | AB      |
| )      |       | AB-     |
| *      | *     | AB-     |
| (      | *(    | AB-     |
| C      | *(    | AB-C    |
| +      | *(+   | AB-C    |
| D      | *(+   | AB-CD   |
| )      | *     | AB-CD+  |
| END    |       | AB-CD+* |

Postfix: AB-CD+\* | Prefix: \*-AB+CD

Expression 2: (A+B)\*C-D/E

**Postfix:**  $AB+C*DE/-$   
**Prefix:**  $-*+ABC/DE$

**Expression 3:**  $(A+B)/(C+D)-(D*E)$

**Postfix:**  $AB+CD+ / DE*-$   
**Prefix:**  $- / + AB+CD*DE$

**Expression 4:**  $A-(B/C+(D\%E*F)/G)*H$

**Postfix:**  $ABC/DEF\%*G/+H*-$   
**Prefix:**  $-A*+ / BC / * \% DEFGH$

**Expression 5:**  $(A+B)*C$

**Postfix:**  $AB+C*$   
**Prefix:**  $*+ABC$

**Expression 6:**  $(A-B/C)*(A/K-L)$

**Postfix:**  $ABC/-AK/L-*$   
**Prefix:**  $*-A/BC-/AKL$

**Expression 7:**  $(A+B)*C$

**Postfix:**  $AB+C*$   
**Prefix:**  $*+ABC$

**Expression 8:**  $(f-g)*((a+b)*(c-d)/e)$

**Postfix:**  $fg-ab+cd-*e/*$   
**Prefix:**  $*-fg/*+ab-cde$